

NGUYEN Luan.

Conception de votre base de données

- Expression des besoins par rapport aux contenus imaginés

Mon site web consiste à la vente de vinyles, on peut trouver comme informations le prix, la durée, l'artiste associé, les sons, des commentaires ou encore le genre. Il me faut donc au minimum 4 pages : une page d'accueil, une page avec tous les vinyles, une page avec les artistes et une avec les genres musicaux. Je vais avoir besoin d'utiliser comme langages de programmation HTML, CSS, JS, PHP, TWIG et SQL.

- Réflexion sur un dictionnaire des données (typage et contraintes des données)

La construction du dictionnaire des données a nécessité une réflexion attentive sur le typage des champs et les contraintes à appliquer afin d'assurer la fiabilité et la cohérence de la base. Chaque type de donnée a été choisi selon la nature des informations à stocker : par exemple, VARCHAR pour les noms ou les titres, TEXT pour les contenus longs comme les descriptions ou les avis, DÉCIMAL pour les prix afin de garantir une précision correcte, et DATE ou TIME pour gérer les durées et les dates.

L'ajout de contraintes telles que NOT NULL, DEFAULT, ou UNIQUE permet de limiter les erreurs d'entrée ou d'incohérence dans les enregistrements. Les clés primaires assurent l'unicité de chaque entité (artiste, vinyle, avis...), tandis que les clés étrangères garantissent l'intégrité des relations entre les tables. La mise en place d'une clé primaire composée dans la table de liaison vinyle_avis empêche les doublons, évitant ainsi qu'un même commentaire soit associé plusieurs fois au même vinyle.

- Commenter le rôle de chaque entité et son usage sur le site

Artistes :

Rôle :

Contient les informations de chaque artiste (musicien, groupe, chanteur, etc.).

Usage sur le site :

- Affichage des fiches artistes avec nom, photo et biographie.
- Permet aux utilisateurs de naviguer entre les artistes et leurs vinyles.
- Point d'entrée pour découvrir la discographie d'un artiste.

Genres :

Rôle :

Répertorie les styles musicaux (rap, trap, R&B, etc.).

Usage sur le site :

- Filtrage des vinyles par genre dans la boutique.

- Éventuelle mise en avant des genres populaires (grâce au champ popularité).

Vinyles :

Rôle :

Représente chaque disque vinyle disponible sur le site.

Usage sur le site :

- Page produit complète avec titre, prix, durée, couverture, etc.
- Accessible depuis les pages vinyles, artistes et genres.

Sons :

Rôle :

Liste les morceaux contenus dans chaque vinyle.

Usage sur le site :

- Affichage des pistes sur la fiche vinyle.
- Lecture d'extraits (si intégré avec un player).
- Indication de popularité avec nombre_ecoute.

Avis

Rôle :

Contient les commentaires des utilisateurs sur les vinyles.

Usage sur le site :

- Affichage des avis utilisateurs sur les pages vinyles.
- Permet à un client de partager son expérience.
- Contient comme informations l'auteur, la date et le contenu du commentaire

Vinyle_avis :

Rôle :

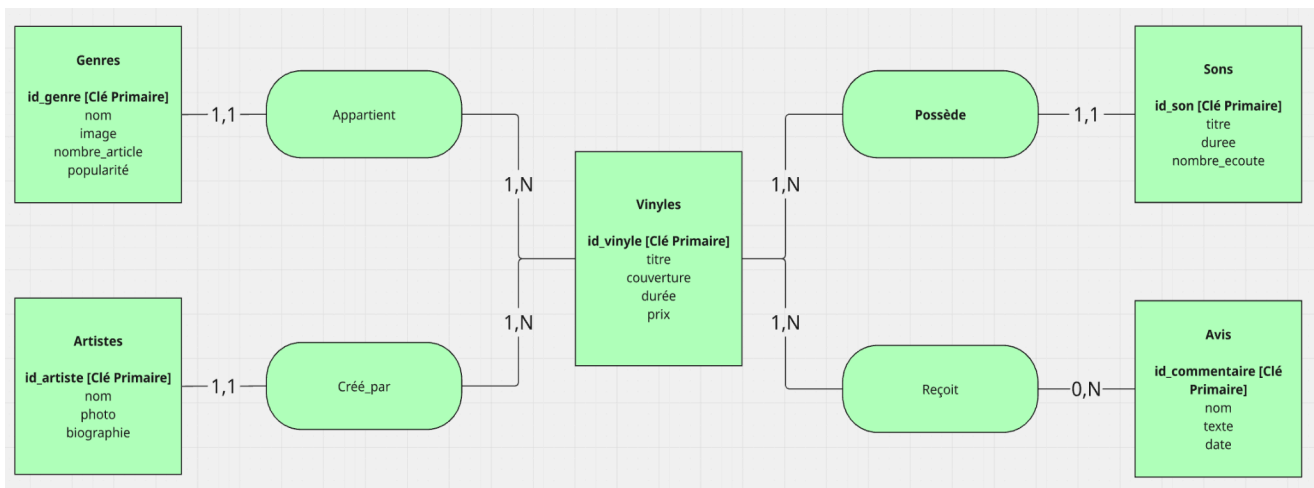
Table de liaison entre les vinyles et les avis. Un vinyle peut posséder plusieurs avis.

Usage sur le site :

- Permet d'associer plusieurs commentaires à un vinyle.
- Garantit que chaque commentaire est rattaché au bon vinyle.

Schéma MCD de votre base de données

- Placer votre schéma ici



- Décrivez en quelques mots simples les tables et leur implémentation avec MySQL

La base de données a été conçue pour structurer les éléments essentiels d'un site web musical. Elle est implémentée avec MySQL, en utilisant des types de données adaptés (INT, VARCHAR, TEXT, DATE, etc.), ainsi que des clés primaires et clés étrangères pour garantir l'intégrité et la cohérence des informations.

Voici une brève description des tables utilisées :

- artistes : stocke les informations relatives aux artistes (nom, photo, biographie).
- genres : répertorie les genres musicaux avec un nom, une description, une image et des indicateurs comme la popularité.
- vinyles : représente les disques vinyles proposés sur le site, avec leur titre, couverture, prix, durée, et des liens vers leur artiste et leur genre.
- sons : contient les morceaux présents sur chaque vinyle, avec leur titre, durée et nombre d'écoutes.
- avis : enregistre les commentaires des utilisateurs avec leur nom, texte et date de publication.
- vinyle_avis : table de liaison permettant d'associer plusieurs avis à un même vinyle, tout en évitant les doublons grâce à une clé primaire composite.

Chaque table est reliée aux autres par des relations logiques via les clés étrangères, ce qui permet une navigation fluide et une gestion cohérente des données.

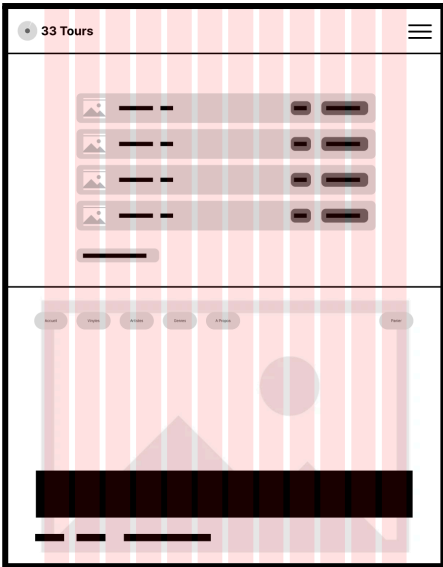
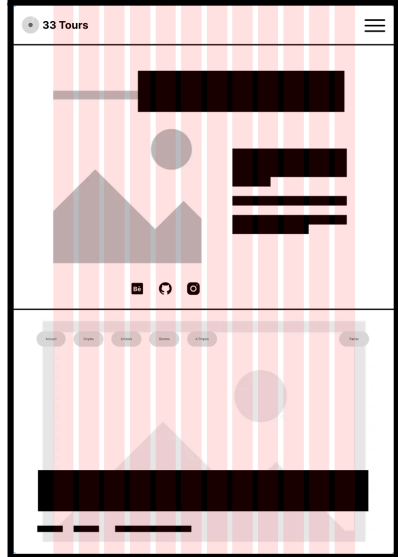
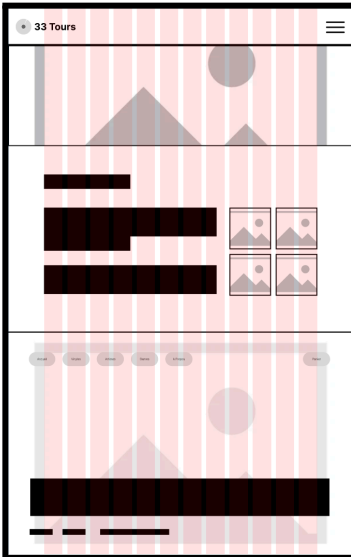
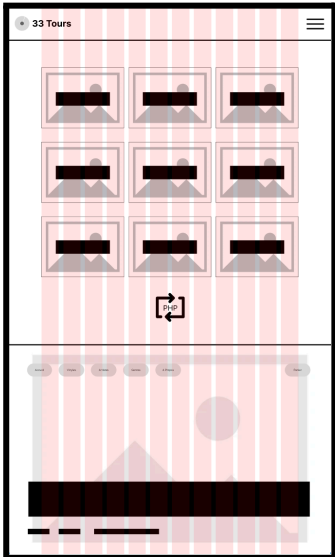
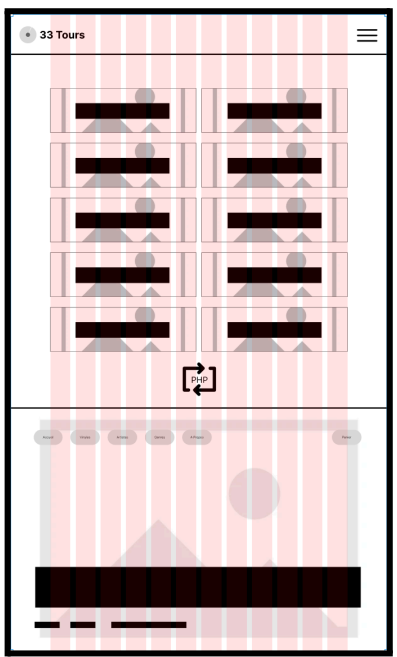
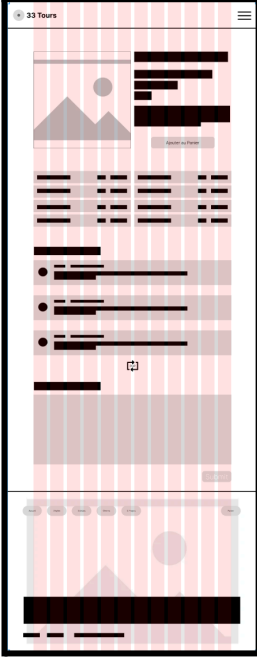
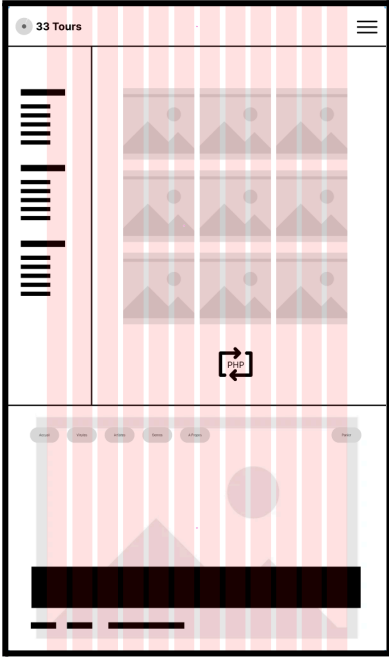
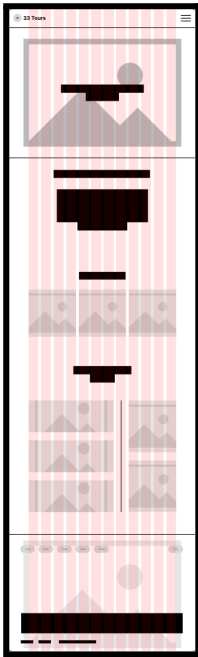
Maquettes et modèles de page

- Rappel du lien : vers la maquette Figma de votre site

<https://www.figma.com/design/gg3K9t4jVxZyRZXaSDFAum/Wireframes-SAE203?node-id=96-816&t=PsqRGbtJx52KAzT-1>

- Lister les modèles de page (fichiers twig) et leur organisation (qui étends qui)
- Base.twig
 - confirmation_commande.twig
 - panier.twig
 - propos.twig
 - vinyles.twig
 - artistes.twig
 - genres.twig
 - detail_vinyle.twig
 - detail_artiste.twig

- detail_genre.twig
- Vous pouvez insérer ici des captures d'écrans de vos maquettes (avec une taille raisonnable = max 50% de la largeur de la page)



- Pour chaque page indiquer les données utilisées pour l’affichage (à côté des captures) (quels types de données ? une variable unique ou un tableau ?)
- Base : \$vinyles, \$genres, \$artistes pour les images et les noms
 - confirmation_commande : rien de particulier
 - panier : \$vinyles pour les couvertures et noms
 - propos : rien de particulier
 - vinyles : \$vinyles pour couvertures et noms
 - artistes : \$artistes pour photos et noms
 - genres : \$genres pour les images et noms
 - detail_vinyle : \$vinyles, \$genres, \$artistes
 - detail_artiste : \$vinyles, \$artistes pour nom, biographie, couvertures et noms de vinyles / artiste
 - detail_genre : \$vinyles, \$genres pour nom, résumé, couvertures et noms de vinyles / genre

Arborescence et fichiers PHP

- Lister les fichiers PHP utilisés et leur organisation
 - index.php
 - vinyles.php
 - artistes.php
 - genres.php
 - propos.php
 - detail_vinyle.php
 - modifier_vinyle.php
 - ajouter_commentaire.php
 - detail_artiste.php
 - detail_genre.php
 - panier.php
 - ajouter_panier.php
 - supprimer_panier.php
 - valider_commande.php
 - confirmation_commande.php
- Pour chaque fichiers indiquer les requêtes utilisées et les paramètres (variable GET) et les fichiers twig utilisé

Dans le fichier `index.php`, trois requêtes SQL récupèrent toutes les données des tables `vinyles`, `artistes` et `genres`, sans utiliser de variable `GET`. Les résultats sont envoyés au template `base.twig`, qui affiche le contenu.

Dans le fichier `vinyles.php`, une seule requête SQL est utilisée pour récupérer la liste complète des vinyles, avec les noms des artistes et des genres associés. Elle utilise des jointures (`LEFT JOIN`) entre les tables `vinyles`, `artistes` et `genres`. Aucune variable `GET` n’est utilisée ici. Le

fichier Twig utilisé pour l'affichage est `vinyles.twig`, auquel on transmet uniquement la variable `vinyles` contenant les résultats de la requête.

Dans le fichier `artistes.php`, une requête SQL récupère tous les artistes depuis la table `artistes`, sans utiliser de variable `GET`. Les données sont envoyées au template `artistes.twig` pour l'affichage.

Dans le fichier `genres.php`, la variable `GET id` est récupérée puis convertie en entier, mais elle n'est pas utilisée dans la requête. Une seule requête SQL sélectionne tous les genres depuis la table `genres`. Les données sont envoyées au template `genres.twig` pour l'affichage.

Dans le fichier `propos.php`, aucune requête SQL ni variable `GET` n'est utilisée. Le template `propos.twig` est simplement affiché sans données.

Dans le fichier `detail_vinyle.php`, la variable `GET id` est utilisée pour identifier un vinyle. Trois requêtes SQL sont exécutées : la première récupère les détails du vinyle avec son artiste et genre, la deuxième récupère les commentaires associés, et la troisième récupère les sons liés au vinyle. Les résultats sont transmis au template `detail_vinyle.twig` pour affichage.

Dans le fichier `detail_artiste.php`, la variable `GET id` est utilisée pour identifier un artiste. Deux requêtes SQL sont exécutées : la première récupère les informations de l'artiste, la seconde ses vinyles. Les données sont envoyées au template `detail_artiste.twig` pour l'affichage.

Dans le fichier `detail_genre.php`, la variable `GET id` sert à sélectionner un genre. Deux requêtes SQL récupèrent les détails du genre et les vinyles associés. Les données sont passées au template `detail_genre.twig` pour affichage.

Dans le fichier `panier.php`, la session est utilisée pour récupérer les IDs des vinyles ajoutés au panier via `$_SESSION['panier']`. Une requête SQL sélectionne ces vinyles avec une clause `IN`. Les résultats sont envoyés au template `panier.twig` pour affichage.

Dans le fichier `ajouter_panier.php`, l'ID du vinyle est récupéré via `POST`. Si cet ID n'est pas déjà dans la session `panier`, il y est ajouté. Un message de confirmation est stocké en session, puis l'utilisateur est redirigé vers la page du vinyle. Sinon, il est redirigé vers l'accueil.

Dans le fichier `supprimer_panier.php`, l'ID du vinyle est récupéré via `GET` puis retiré de la session `panier` si présent. L'utilisateur est ensuite redirigé vers la page du panier.

Dans le fichier `ajouter_commentaire.php`, aucune variable `GET` n'est utilisée. Les données sont récupérées via la méthode `POST` : l'identifiant du vinyle, le nom, le prénom et le commentaire de l'utilisateur. Ces données sont nettoyées avec `htmlspecialchars` et `strip_tags` pour éviter les injections et failles XSS. Trois requêtes SQL sont ensuite exécutées : la première insère le commentaire dans la table `avis`, la seconde récupère l'ID auto-incrémenté du commentaire, et la troisième crée l'association entre ce commentaire et le vinyle concerné dans la table `vinyle_avis`. La session est utilisée pour stocker un message de confirmation. À la fin du traitement, l'utilisateur est redirigé vers la fiche du vinyle (`detail_vinyle.php?id=`). Aucun template Twig n'est utilisé, car le fichier sert uniquement à traiter et rediriger.

Dans le fichier `valider_commande.php`, la session est utilisée pour accéder au panier de l'utilisateur. Si la clé `$_SESSION['panier']` existe, elle est supprimée afin de vider le panier. Aucun accès à la base de données ni affichage de page n'est effectué dans ce script. Un message de confirmation est ensuite enregistré en session, puis l'utilisateur est redirigé vers la page `confirmation_commande.php`.

Dans le fichier `confirmation_commande.php`, le moteur de template Twig est initialisé via le fichier `include/twig.php`. Le message de confirmation, stocké précédemment en session, est récupéré dans la variable `$message` et transmis au template `confirmation_commande.twig` pour affichage. Après le rendu, ce message est supprimé de la session. Aucune requête SQL n'est utilisée ici.

Dans le fichier `modifier_vinyle.php`, l'identifiant d'un vinyle est récupéré via la variable GET `id`. Si un formulaire POST est soumis, les nouvelles informations du vinyle (titre, prix, genre, artiste) sont nettoyées puis utilisées dans une requête SQL de mise à jour (`UPDATE vinyles`). Un message de confirmation est ensuite enregistré en session et l'utilisateur est redirigé vers la page de détail du vinyle. En l'absence de soumission POST, les informations actuelles du vinyle sont récupérées via une requête SQL (`SELECT *`) ainsi que les listes complètes des genres et artistes (pour les menus déroulants). L'affichage de la page se fait sans Twig, en HTML pur avec des balises PHP.

40

Comment avez-vous produit des pages fluides incluant un balisage sémantique efficace ?

- Qu'est-ce que cela signifie pour moi ?

Produire des pages fluides avec un balisage sémantique efficace signifie que je suis capable de concevoir des sites web à la fois lisibles, structurés et accessibles. Cela montre que je maîtrise l'utilisation des balises HTML5 appropriées pour organiser le contenu de manière logique et compréhensible, aussi bien pour les utilisateurs que pour les moteurs de recherche ou les technologies d'assistance. En rendant mes pages fluides, je m'assure également qu'elles s'adaptent correctement à différentes tailles d'écran, ce qui améliore l'expérience utilisateur sur tous les appareils. Pour moi, cela reflète une approche professionnelle du développement web, où la qualité du code, l'ergonomie et l'accessibilité sont des priorités essentielles.

- Quelles technologies ai-je utilisé ?

Si on parle de langages de programmations et de style, j'ai utilisé du HTML, CSS, JAVASCRIPT, PHP, TWIG et SQL. Sinon pour les wireframes j'ai d'abord utilisé du papier + crayon puis lorsque j'étais sûr de mon idée je les ai réalisées sur Figma.

- Comment me suis-je assuré que les pages intégrées respectaient ma maquette ?

J'ai comparé tout au long de la conception si mon site a respecté la mise en forme que j'ai faite sur les wireframes tels que les marges de côtés ou encore la disposition des textes et des images.

Comment avez-vous mis en ligne votre site en utilisant une solution d'hébergement standard ?

- Comment avez-vous procédé ?

J'ai utilisé Plesk comme hébergeur : J'ai mis mes fichiers dans le httpdocs , j'ai créé un sous-domaine avec comme chemin mon dossier contenant mon site, j'ai créé une base de données et j'ai inséré mon fichier sql contenant les tables et données puis je l'ai lié à mon sous-domaine ce qui m'a ensuite permis d'accéder à mon site. J'ai aussi changé l'identifiant et le mot de passe du fichier config.php avec ceux créés lors de la base de données.

- Comment avez-vous assuré la sécurité de l'hébergement ?

J'ai créé des fichiers .htaccess avec les lignes suivantes :

Options -Indexes

ErrorDocument 404 /SAE203_NGUYEN_Luan_V3/404.html

ErrorDocument 403 /SAE203_NGUYEN_Luan_V3/403.html

Cela permet de bloquer l'accès au dossier tel que templates, include ou encore vendor en redirigeant en plus vers la page 403.html avec comme message "Vous n'avez pas la permission d'accéder à cette ressource." et si l'utilisateur cherche quelque chose d'introuvable, celui-ci va être redirigé vers la page 404.html avec comme message " Oups ! La page demandée est introuvable."

En plus de ça, j'ai protégé mon site des injections sql grâce à la protection de caractères de type htmlspecialchars et strip_tags dans mes formulaires.